UNIVERSITÉ IBA DER THIAM DE THIÈS
Direction Technique & Pôle Développement • Campus Market

STANDARD D'INGÉNIERIE LOGICIELLE

Guide Officiel du Workflow GitHub

Gestion des Branches, Déploiements Vercel, Intégrité Supabase et Sécurité Anti-Secrets pour l'Équipe Étudiante.

Projet : **Campus Market (UIDT)** Production : **campusmarket-uidt.com** Version : **1.0 (Novembre 2026)**

📚 Table des Matières

1. Philosophie & Architecture Globale du Workflow **Page 2**
2. Stratégie de Branches (GitHub Flow & Vercel Preview) **Page 3**
3. Règle des Commits Conventionnels (Conventional Commits) **Page 4**
4. Sécurité Fondamentale : Règle "Zéro Secret en Dur" **Page 5**
5. Règle d'Or des Migrations Additives Supabase (SQL) **Page 6**
6. Automatisation CI/CD (GitHub Actions) **Page 7**
7. Modèles de Pull Request, Revue & Recette Mobile **Page 8**
8. Guide Pratique de l'Étudiant Développeur (Cheat Sheet) **Page 9**

1. Philosophie & Architecture Globale

Campus Market est la plateforme de commerce électronique officielle des étudiants et commerçants de l'Université Iba Der Thiam de Thiès (UIDT). Déployée en production réelle sur `campusmarket-uidt.com`, elle accueille chaque jour des centaines de transactions d'achat et des paiements mobiles (Wave, Orange Money).

> **🎯 Objectif Pédagogique & Professionnel**
>
> Ce guide formalise les méthodes de travail adoptées par les meilleures équipes d'ingénierie tech mondiales. L'objectif est double :
> **sanctualiser la production**
> contre toute régression et
> **former les étudiants**
> aux standards stricts de l'industrie (CI/CD, revues de code, sécurité, migrations additives).

Schéma Directeur du Cycle de Vie d'une Fonctionnalité



Flux continu : Aucun code n'arrive en production sans avoir franchi les 3 sas de sécurité

2. Stratégie de Branches (GitHub Flow Moderne)

Sur les projets Web modernes déployés via Vercel, l'ancien modèle lourd "GitFlow" (avec branches `release`, `hotfix`, `develop` enchevêtrées) est abandonné au profit d'un **GitHub Flow agile et robuste** :

Branche	Règle de Protection	Rôle & Environnement Associé
`main`	VERROUILLÉE	**Production absolue.** Connectée au domaine public `campusmarket-uidt.com`. Push direct formellement interdit (`git push origin main` est bloqué). Toute modification transite obligatoirement par une Pull Request validée.
`feature/*`	Libre pour l'auteur	Branche de travail éphémère pour développer une nouvelle fonctionnalité (ex: `feature/v2-seller-badges`). Chaque PR ouverte génère automatiquement une URL de prévisualisation Vercel.
`fix/*`	Libre pour l'auteur	Branche de correction de bogue (ex: `fix/safari-keyboard-overlap`). Doit comporter un test prouvant la correction.
`hotfix/*`	Prioritaire	Branche d'urgence pour incident critique en production. Tirée depuis `main`, testée immédiatement et fusionnée en priorité.

> ⚠️ **Nettoyage de l'Ancienne Branche master**
> Le dépôt distant contient un résidu historique `origin/master`. Pour éviter qu'un étudiant ne clone ou ne pousse sur cette branche obsolète, la branche par défaut officielle de GitHub est `main`.

Règles de Protection de Branche à Activer sur GitHub

Dans **Settings > Branches > Add rule** sur le dépôt GitHub de Campus Market :

- **Branch name pattern** : `main`
- ☑️ **Require a pull request before merging** : Bloque tout push direct sans PR.
- ☑️ **Require approvals** : Au minimum 1 approbation requise par un collègue développeur.
- ☑️ **Require status checks to pass before merging** : Le job CI `validate` doit être au vert.
- ☑️ **Require linear history / Squash and merge** : Fusionne les commits en un seul bloc clair dans l'historique de production.
- ☑️ **Do not allow bypassing the above settings** : S'applique même aux administrateurs pour prévenir les étourderies.

3. Règle des Commits Conventionnels

L'équipe de Campus Market applique la norme internationale **Conventional Commits v1.0.0**. Chaque message de commit doit expliciter la *nature* et la *portée* du changement :

```
<type>(<périmètre>): <description claire et concise à l'infinitif>
```

Type	Périmètres Fréquents	Exemple de Message Valide
FEAT	`checkout`, `catalog`, `orders`, `push`	`feat(checkout): ajout du badge Wave et Orange Money`
FIX	`safari`, `auth`, `routing`, `pwa`	`fix(auth): fiabiliser le renouvellement du token Supabase`
SEC	`security`, `rpc`, `rls`, `secrets`	`fix(security): supprimer la clé privée VAPID écrite en clair`
DOCS	`readme`, `workflow`, `api`	`docs(workflow): guide de test mobile sur Vercel Preview`
STYLE	`mobile`, `theme`, `safe-area`	`style(mobile): conformité des cibles tactiles WCAG 44px`
TEST	`seo`, `orders`, `compression`	`test(orders): vérification du recalcul des prix côté serveur`
REFACTOR	`router`, `services`, `state`	`refactor(services): découplage du client Supabase`

> 🚫 **Commits Vagues Strictement BANNIS**
> Les messages tels que `modifs`, `changement`, `fix`, `wip`, `update` ou `test` sont formellement interdits. Ils rendent l'analyse post-incident et les rollbacks impossibles.

Règle de l'Atomicité des Commits

Un bon commit ne réalise qu'**une seule tâche logique**. Ne mélangez jamais dans un même commit un correctif de style CSS, une modification de schéma SQL et une refonte de script d'authentification.

4. Sécurité Fondamentale : Règle "Zéro Secret en Dur"

En tant que plateforme gérant des fonds d'abonnements marchands et des informations étudiantes, la sécurité n'est pas négociable.

> 🚨 **RÈGLE ABSOLUE DE SÉCURITÉ**
>
> Aucun secret serveur, aucune clé privée et aucun token d'authentification ne doit **JAMAIS** être commité dans le dépôt Git, même temporairement ou en tant que valeur de secours (fallback).

Variable Sensible	Localisation Requise	Danger en cas de fuite Git
`SUPABASE_SERVICE_ROLE_KEY`	Vercel Settings & `.env.local`	**Critique.** Contourne toutes les sécurités RLS PostgreSQL et donne un accès administrateur total à la base de données.
`VAPID_PRIVATE_KEY`	Vercel Settings & `.env.local`	**Élevé.** Permet à un pirate d'usurper l'identité du serveur et d'émettre des notifications push frauduleuses.
`SASPAY_SECRET_KEY`	Vercel Settings & `.env.local`	**Financier.** Permet de détourner ou valider arbitrairement des transactions Wave et Orange Money.
`SASPAY_WEBHOOK_SECRET`	Vercel Settings & `.env.local`	**Financier.** Permet d'injecter de fausses notifications IPN pour valider des abonnements sans paiement réel.

Principe Fail-Fast : Zéro Fallback de Complaisance

L'écriture de fallbacks de complaisance est strictement interdite dans le code source :

```
// ❌ INTERDIT (Fuite de clé en clair dans le code) :
const privateKey = process.env.VAPID_PRIVATE_KEY || 'MIGHAgEAMBMGByqGSM49AgEGPC...';

// ✅ OBLIGATOIRE (Échec immédiat si la configuration d'environnement est manquante) :
const privateKey = process.env.VAPID_PRIVATE_KEY;
if (!privateKey) {
  throw new Error("Variable d'environnement VAPID_PRIVATE_KEY requise !");
}
```

Garde-Fou Automatisé : Le Scanner Anti-Secrets

Le projet intègre le script `scripts/verify-no-secrets.js`. Exécuté en local et automatiquement par la CI GitHub Actions, il analyse les fichiers sources et bloque immédiatement le build s'il repère une tentative d'injection de secret.

5. Règle d'Or des Migrations Additives Supabase

Le site étant en production publique active sur `campusmarket-uidt.com`, la base de données PostgreSQL de Supabase contient les profils, les commandes réelles et les boutiques des étudiants.

> 🛡️ **Principe de Migration Additive (Strictement Rétrocompatible)**
> Toute évolution de la base de données doit être additive. Il est formellement interdit d'exécuter un script détruisant ou modifiant brutalement des colonnes en production sans période de transition.

Les 4 Lois des Migrations SQL à l'UIDT

1. **Ajout de colonnes tolérantes** : Toute nouvelle colonne doit avoir une valeur par défaut (`DEFAULT`) ou accepter les valeurs nulles (`NULL`) pour ne pas briser la version du site en cours d'exécution.
2. **Interdiction des `DROP` directs** : Les instructions `DROP TABLE` ou `DROP COLUMN` sont proscrites des scripts de mise à jour courants.
3. **Vérification d'existence (Idempotence)** : Toujours utiliser `CREATE TABLE IF NOT EXISTS` , `ADD COLUMN IF NOT EXISTS` et `CREATE OR REPLACE FUNCTION` .
4. **Exécution contrôlée dans le SQL Editor** : Les migrations ne sont jamais exécutées automatiquement à l'aveugle. Elles sont revues dans la Pull Request et appliquées manuellement via la console Supabase par le SuperAdmin.

Sécurité & Immunité Entre Pairs du Rôle SuperAdmin

Sur l'interface de gestion (`SuperAdminDashboard.js`), tous les administrateurs sont strictement pairs et égaux. Aucun SuperAdmin ne peut suspendre, rétrograder ou supprimer un autre SuperAdmin depuis le Web.

> L'attribution ou la révocation des privilèges `superadmin` s'effectue
> **exclusivement au niveau de la base de données**
> via le script SQL `database/bootstrap_superadmin.sql` dans le SQL Editor de Supabase.

6. Automatisation CI/CD (GitHub Actions)

Pour soulager les étudiants de vérifications manuelles fastidieuses, le dépôt est doté du workflow GitHub Actions `.github/workflows/ci.yml`.

Déclenchement & Performance

- **Événements déclencheurs** : Chaque ouverture ou mise à jour de Pull Request vers `main`, et chaque push de validation.
- **Temps d'exécution garanti** : Moins de **45 secondes** sur les serveurs GitHub Actions (Ubuntu 22.04 LTS + Node 20 LTS).
- **Stratégie de cache** : Cache npm intelligent pour accélérer l'installation des dépendances.

Matrice des 3 Contrôles Automatiques

Étape	Commande Exécutée	Critère de Succès
1. Audit Zéro Secret	`node scripts/verify-no-secrets.js`	Aucun motif suspect (clés privées, service_role, fallbacks) détecté.
2. Tests Métier & Sécurité	`npm test`	100% des tests unitaires validés (Sanity Check, SEO, Compresseur d'images, PWA, Routeur URL, Classement Top Shops, Round-Robin, Web Push).
3. Compilation Production	`npm run build`	Génération complète du bundle de production Vite dans `dist/` sans erreurs de syntaxe.

> ⛔ **La Règle du Feu Rouge**
>
> Si la CI échoue (croix rouge ❌ sur GitHub),
> **la Pull Request ne peut en aucun cas être fusionnée**
> . L'étudiant doit corriger l'erreur localement et repousser sur sa branche.

7. Modèles de PR, Revue & Recette Mobile

Le commerce étudiant se vit sur smartphone. Plus de 90% des consultations de Campus Market ont lieu depuis un iPhone ou un terminal Android.

Vercel Preview : Votre Environnement de Test Réel

Dès qu'une Pull Request est ouverte sur GitHub, Vercel génère une URL de test éphémère (ex: `campus-market-git-feature-xyz.vercel.app`). Cette URL est immédiatement accessible depuis votre téléphone portable.

Checklist Mobile Obligatoire (Template de PR)

Avant d'approuver une Pull Request, le relecteur et l'auteur doivent valider ces points sur leur smartphone :

- ☑️ **Cibles tactiles WCAG** : Boutons et icônes d'au moins 44x44px, faciles à presser au pouce.
- ☑️ **Gestion des Safe Areas** : Pas de masquage de l'interface par la Dynamic Island d'iOS ou la barre de navigation Android.
- ☑️ **Clavier virtuel** : L'ouverture du clavier ne déplace pas brutalement la barre de navigation fixe.
- ☑️ **PWA & Mode Hors-ligne** : Le bandeau d'installation reste discret et conforme.

Règles de Gouvernance (CODEOWNERS)

Le fichier `.github/CODEOWNERS` assigne automatiquement les relectures prioritaires aux leads techniques pour les périmètres critiques :

```
# Sécurité, API serverless et Base de données
/api/            @NdiayeDigital
/database/       @NdiayeDigital
/.github/        @NdiayeDigital
```

8. Guide Pratique de l'Étudiant Développeur (Cheat Sheet)

Voici la séquence exacte des commandes à exécuter au quotidien pour travailler sereinement sur Campus Market :

Étape 1 : Mettre à jour sa copie locale

```
# Se positionner sur main et récupérer les dernières nouveautés de l'équipe
git checkout main
git pull origin main
```

Étape 2 : Créer une branche de travail dédiée

```
# Créer et basculer sur une nouvelle branche feature ou fix
git checkout -b feature/nom-de-ma-fonctionnalite
```

Étape 3 : Développer et tester en temps réel

```
# Lancer Vite avec accès réseau pour tester sur son smartphone connecté au Wi-Fi
npm run dev -- --host

# L'adresse affichée (ex: http://192.168.1.45:5173) s'ouvre dans Safari ou Chrome mobile !
```

Étape 4 : Lancer le contrôle consolidé avant de commit

```
# Commande tout-en-un vérifiant les secrets, les tests et le build :
npm run check
```

Étape 5 : Enregistrer son travail avec un commit conventionnel

```
git status
git add .
git commit -m "feat(catalog): ajout du filtre par categorie et prix"
```

Étape 6 : Publier sa branche et ouvrir la Pull Request

```
# Envoyer sa branche sur GitHub
git push -u origin feature/nom-de-ma-fonctionnalite

# Ouvrir le lien affiché dans le terminal pour créer la Pull Request sur GitHub.
# Remplir le template de PR, vérifier le passage de la CI et tester le lien Vercel Preview !
```

> 🎓 **En Résumé**
>
> En respectant ce cycle simple :
> **Branche → Tests → Commit Clair → PR → Test Mobile → Merge**
> , vous développez comme les meilleures équipes d'ingénierie logicielle de la Silicon Valley et d'Europe tout en protégeant le campus de l'UIDT !